Software Design and Analysis (SDA)

Title:

Karachi Smart Transit System (KSTS)

Submitted by:

Ghulam Mujtaba Qureshi 24K-0535

Adil Saeed 24K-0705

Haseem Samo 24K-0666

Zaid Faraz 24K- 0636

Course: CS3004 Software Design and Analysis
Instructor: Mr. Jibran Rasheed Khan
Date: April 29, 2026

Table of Contents

Chapter 1: Introduction 3
- System Overview 3
- Problem Statement 3
- Scope 4
- Objectives 4
- Users and Stakeholders 5
- System Requirements Summary 5
- Software Process Model 6

Chapter 2: System Analysis and Modeling 7
- Use Case Diagram 7
- Class Diagram 12
- Activity Diagrams 14
- Sequence Diagrams 16
- Deployment Diagram 19
- Timing Diagram 20

Architectural View (4+1 Model) 21
- Logical View 21
- Process View 23
- Development View 24
- Physical View 25
- +1 (Use Case View) 26

References 27

Chapter 1: Introduction

System Overview

The Karachi Smart Transit System (KSTS) is a comprehensive, database-driven software platform designed to model, manage, and optimize public transport services in Karachi, with a primary focus on Karachi Breeze (BRT) and the Peoples Bus Service. The system enables passengers to register, log in, view schedules, book trips, make payments (online or via recharge cards), and receive tickets. Administrators are empowered to manage routes, buses, and drivers, schedule trips, and generate analytical and operational reports.

The KSTS centralizes transport-related data, improves operational efficiency, minimizes scheduling conflicts and seat underutilization, and supports data-driven decision-making for route optimization, service planning, and performance monitoring. The system is designed for scalability and reliability to accommodate multiple buses on the same route and frequent trips while ensuring secure and user-friendly interactions for all stakeholders.

Problem Statement

Karachi's public transport system currently faces fragmented management, lack of centralized data, and reliance on manual trip scheduling, resulting in multiple inefficiencies, including:

- Overlapping trips or underutilized buses, leading to resource wastage.
- Difficulty in monitoring seat availability, which hampers passenger planning and satisfaction.
- Inefficient booking and payment processes, causing delays and errors in ticketing.
- Limited access to analytical insights for administrators, restricting data-driven decision-making for route planning and service optimization.

KSTS aims to resolve these challenges by providing a unified, real-time platform that integrates route management, trip scheduling, passenger booking, ticketing, and payment processing. This system enhances operational efficiency, improves passenger experience, and supports administrators with actionable insights for informed decision-making.

Scope

The system is designed to manage and optimize Karachi's public transport services. The system includes the following functionalities:

- **Passenger Registration and Login:** Secure account creation, authentication, and profile management.
- **Trip Booking and Cancellation:** Easy selection of routes, scheduling, and ticket cancellation options.
- **Seat Management and Ticket Generation:** Allocation of seats and automated generation of digital tickets.
- **Payment Processing:** Support for online payments and physical recharge card transactions.
- **Route, Bus, and Driver Management:** Administrators can define, update, and monitor transport resources.
- **Trip Scheduling:** Automated and manual trip planning to optimize service efficiency.
- **Reporting and Analytics:** Generate operational reports and insights for informed decision-making by administrators.

It does not cover the following, which may be considered for future enhancements:

- Voice-based booking input

Objectives

The primary objectives of the system are to:

1. **Facilitate Efficient Passenger Operations:** Enable passengers to securely register, book trips, manage schedules, and cancel bookings when necessary.
2. **Empower Administrators with Management Tools:** Provide comprehensive functionalities for managing routes, buses, drivers, and schedules effectively.
3. **Ensure Secure and Versatile Payment Handling:** Integrate safe online transactions and physical recharge card options with automated receipts and ticket generation.
4. **Support Data-Driven Decision-Making:** Generate detailed analytics and reports for administrators to monitor performance and optimize operations.
5. **Ensure System Scalability and Reliability:** Support multiple buses on the same route, frequent trips, and concurrent passenger activities without conflicts.

Users and Stakeholders

Primary Users:

- **Passenger:** Registers on the system, searches for trips, books and cancels trips, makes payments (online or via recharge card), and receives tickets.
- **Admin:** Manages routes, buses, drivers, and schedules; monitors system performance; and generates operational and analytical reports.
- **Driver:** Verifies passenger tickets, monitors seat availability, and ensures proper boarding and disembarking procedures.

Stakeholders:

- **Karachi Transport Authorities:** Provide oversight of public transport efficiency, regulatory compliance, and service standards.
- **Payment Gateway Providers:** Facilitate secure online payment processing, reconciliation, and transaction tracking.

System Requirements Summary

Functional Requirements:

- **User Authenticatio** all system users, including passengers, admins, and drivers.
- **Trip Booking and Cancellation:** Passengers can search, book, and cancel trips; system generates tickets upon confirmation.
- **Payment Processing and Confirmation:** Supports online payments and recharge card transactions, with secure transaction verification and receipt generation.
- **Route, Bus, and Driver Management:** Admins can define and modify routes, manage bus assignments, and track driver details.
- **Trip Scheduling:** Automatic scheduling based on route frequency, with optional manual overrides for special cases or adjustments.
- **Reporting and Analytics:** Generate operational, financial, and usage reports for administrators to support decision-making and performance monitoring.

Non-Functional Requirements:

- **Reliability:** System must handle multiple concurrent bookings and transactions without conflicts or data loss.
- **Scalability:** Capable of supporting numerous buses, trips, and users simultaneously without performance degradation.
- **Security:** Ensure secure storage of user data and financial transactions, complying with best practices for authentication and encryption.
- **Usability:** Intuitive and user-friendly interface for passengers, drivers, and administrators, minimizing training requirements and user errors.

Software Process Model

Chosen Model: Iterative/Agile Process Model

Justification:

- The KSTS consists of multiple interdependent modules, including Passenger, Admin, Driver, and Payment Service, with requirements that are expected to evolve over time.
- The Agile/Iterative approach is ideal for this project because it enables:
    - Rapid prototyping and iterative feedback for passenger and administrator workflows.
    - Incremental development of system modules, such as booking, payment, reporting, and analytics.
    - Early testing, validation, and refinement of both database design and user interface components.
- Iterative cycles ensure that core functionalities (trip booking, ticketing, and payment processing) are continuously tested and improved, reducing risk and enhancing system reliability.

Chapter 2: System Analysis and Modeling

Use Case Diagram

Use Case Description Tables

Register/Login

Use Case Name	Register/Login
Use Case Description	Allows users (Passenger, Admin) to create an account or log into the system.
Actors	Passenger (Primary), Admin (Other)
Stakeholders	System Administrator, Registered Users
Description	Provides secure access to system functionalities.
Includes	None
Extends	None
Input	Username, Password, or Registration Details
Pre-conditions	User must have access to the system interface.
Flow of events	1. User opens login/registration page. 2. Enters credentials or registration details. 3. System verifies and grants access.
Alternate	Invalid credentials or multiple failed attempts → account locked.
Post-conditions	User session created and access granted.

Book Trip

Use Case Name	Book Trip
Use Case Description	Allows passengers to select and reserve a trip.
Actors	Passenger (Primary), Payment Service (Other)
Stakeholders	Passengers, Admin
Description	Enables users to plan and confirm travel.
Includes	Handle Payment, Generate Ticket
Extends	Seat Upgrade
Input	Trip details (origin, destination, date, payment info)
Pre-conditions	Passenger must be logged in.
Flow of events	1. Passenger selects trip and timing. 2. System checks seat availability. 3. Passenger confirms and pays. 4. Payment processed. 5. Ticket generated.
Alternate	Payment failure or no available seats → booking canceled.
Post-conditions	Trip booked successfully; ticket stored in passenger account.

Generate Ticket

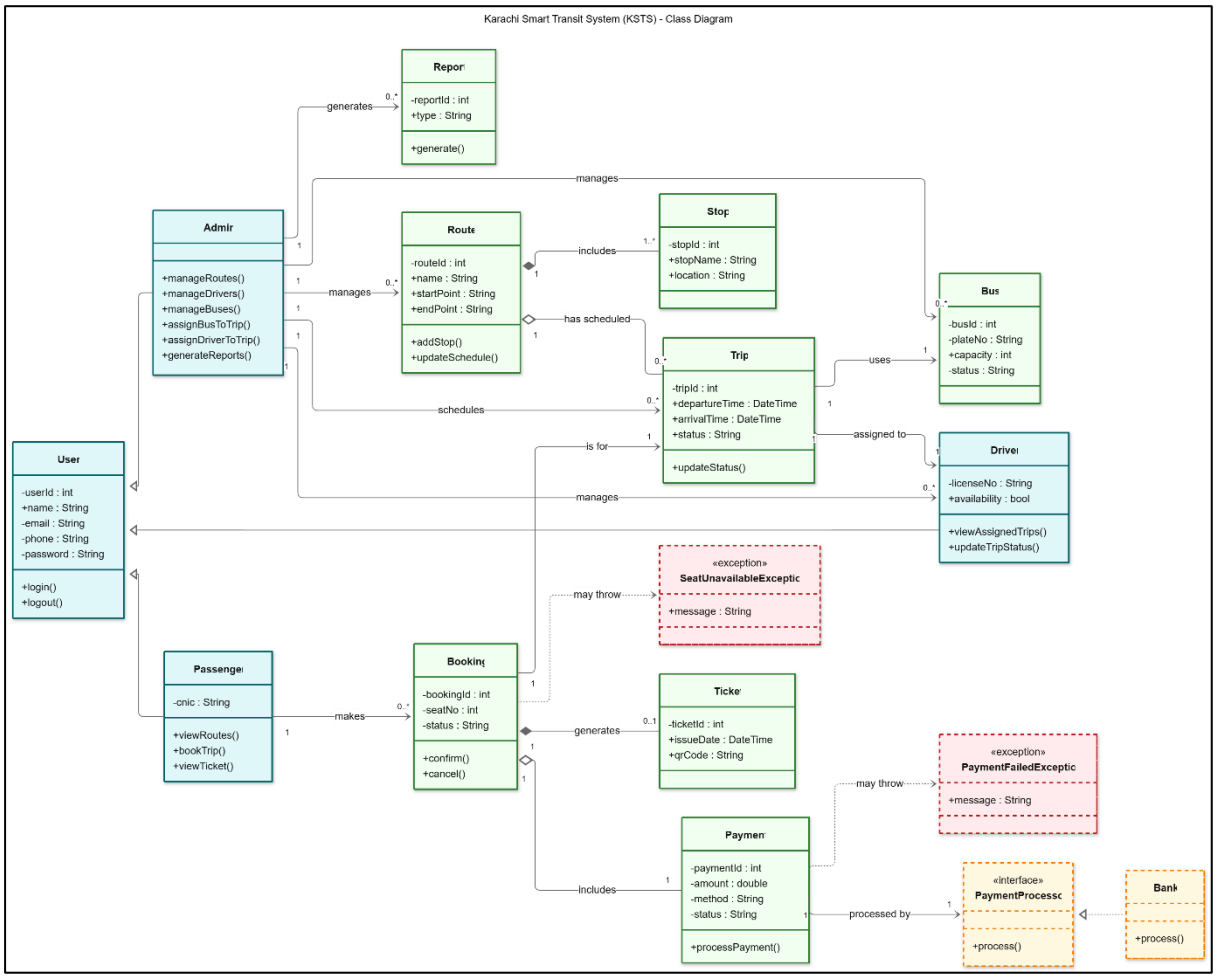
Use Case Name	Generate Ticket
Use Case Description	Produces a digital or printable ticket after booking.
Actors	Passenger (Primary), Admin (Other)
Stakeholders	Passenger, Transit Authority
Description	Confirms passenger's booking and travel authorization.
Includes	None
Extends	None
Input	Booking ID, Payment Confirmation
Pre-conditions	Payment must be successful.
Flow of events	1. System generates ticket details. 2. Ticket assigned to user. 3. Notification sent.
Alternate	Ticket generation error → user notified.
Post-conditions	Ticket generated and available for verification.

Assign Bus and Driver

Use Case Name	Assign Bus and Driver
Use Case Description	Allows admin to assign drivers and buses to routes.
Actors	Admin
Stakeholders	Drivers, Passengers
Description	Ensures each scheduled trip has an assigned driver and vehicle.
Includes	None
Extends	None
Input	Driver ID, Bus ID, Route ID
Pre-conditions	R
Flow of events	1. Admin selects route. 2. Assigns driver and bus. 3. System updates and notifies driver.
Alternate	Driver unavailable → admin selects another.
Post-conditions	Bus and driver successfully assigned.

Class Diagram

a. Classes, Attributes, Methods, Relationships



b. Boundary, Control, Entity Classes (BCE Model)

Boundary

Boundary classes represent user interfaces or interaction points between the system and external actors.

- **PassengerUI:** Interface for passengers to browse routes, book trips, and view tickets.
- **DriverUI:** Interface for drivers to view assigned trips and update trip status.
- **AdminUI:** Interface for administrators to manage routes, buses, trips, and reports.

Control

Control classes manage the flow of logic for specific use cases.

- **BookingController:** Coordinates booking creation, seat assignment, ticket generation, and payment handling.
- **PaymentController:** Manages payment initialization, validation, and selection of processor.
- **TripController:** Handles trip scheduling, driver assignment, and bus assignment.
- **RouteController:** Manages route creation, stop management, and schedule updates.
- **ReportController:** Generates reports requested by the Admin.

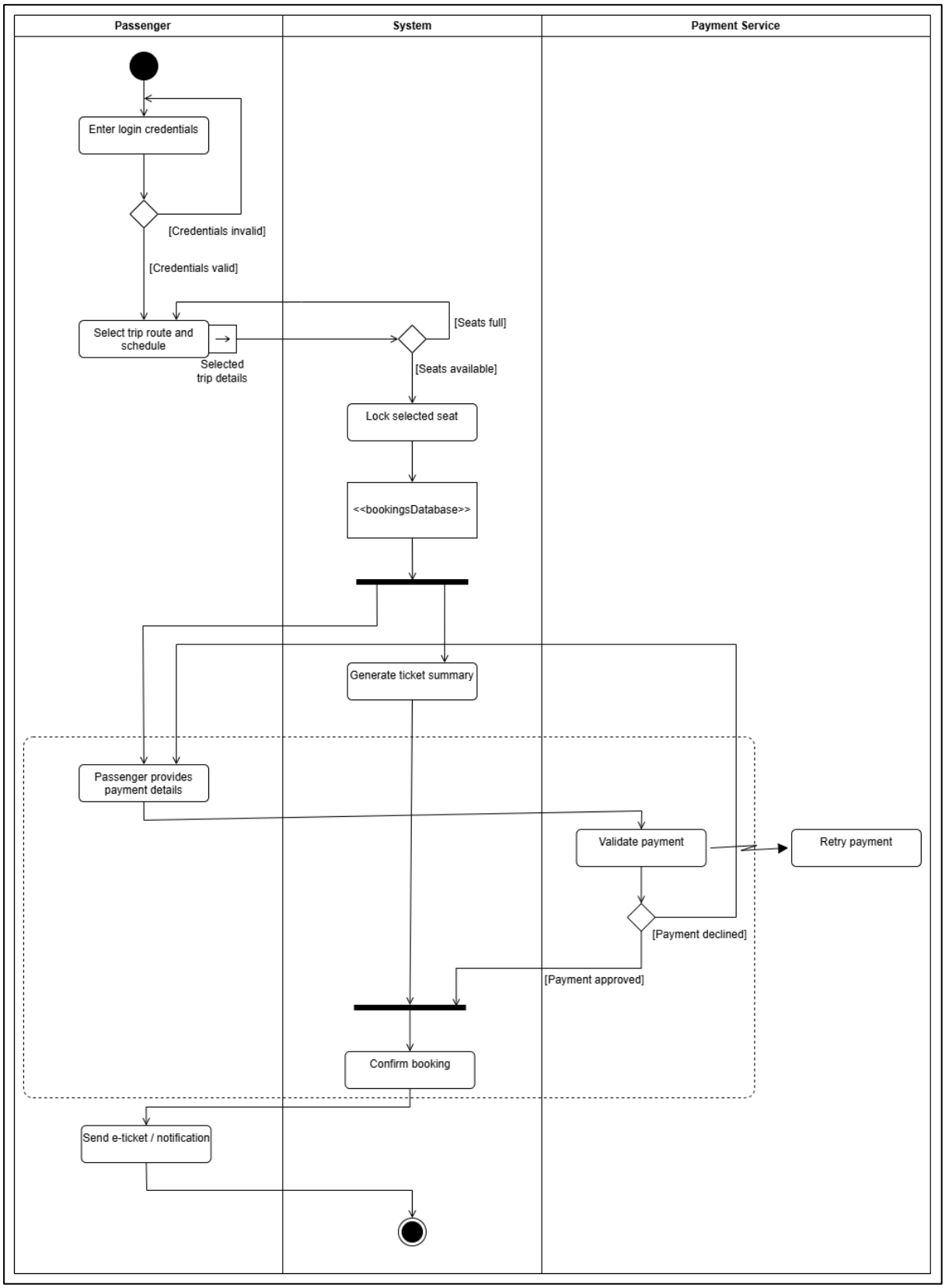
Entity

Entity classes represent persistent domain concepts stored in the system.

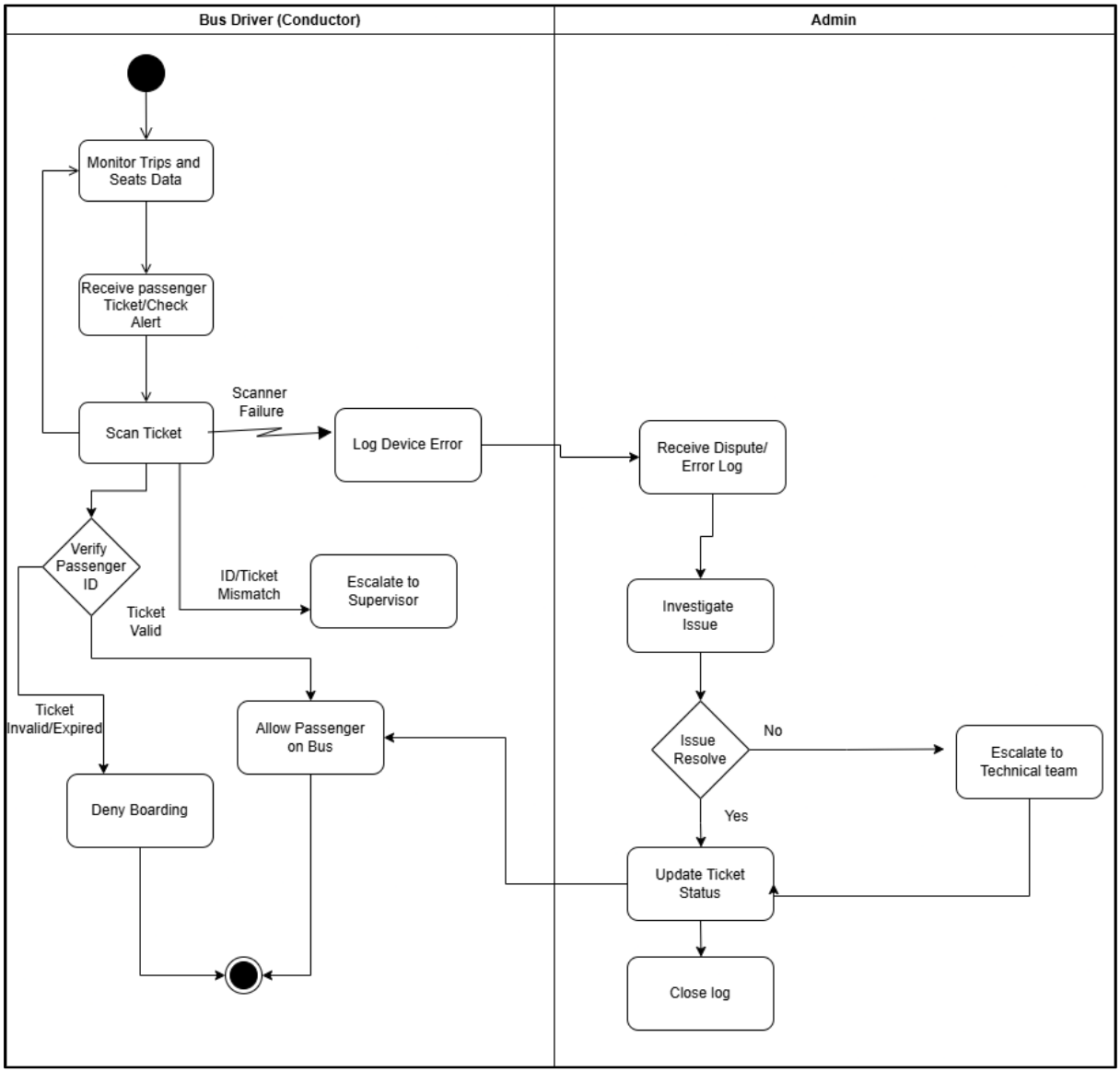
- User
- Passenger
- Driver
- Admin
- Route
- Stop
- Bus
- Trip
- Booking
- Ticket
- Payment
- Report
- PaymentProcessor (interface)
- Bank (implements PaymentProcessor)

Activity Diagrams

Passenger Book Trip

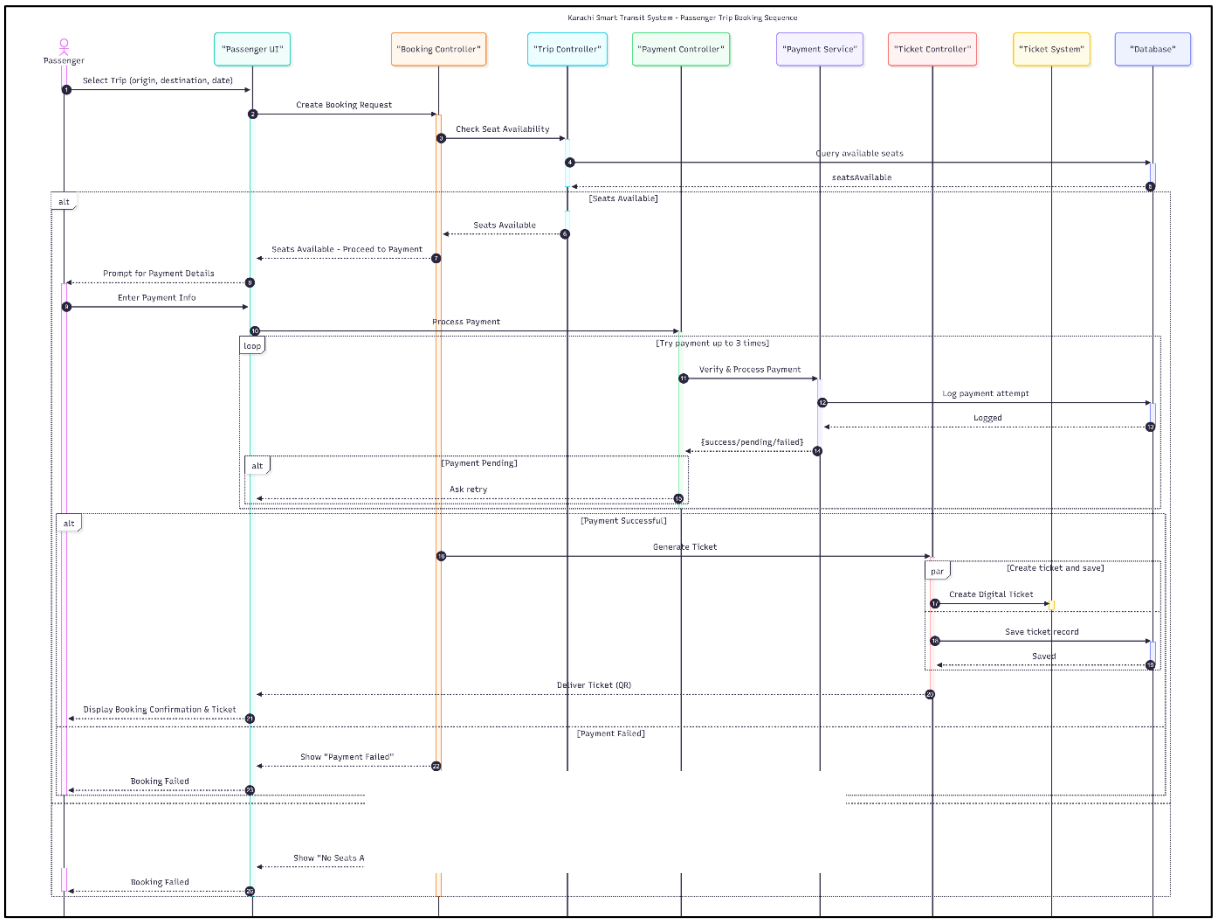


Driver Ticket Validate

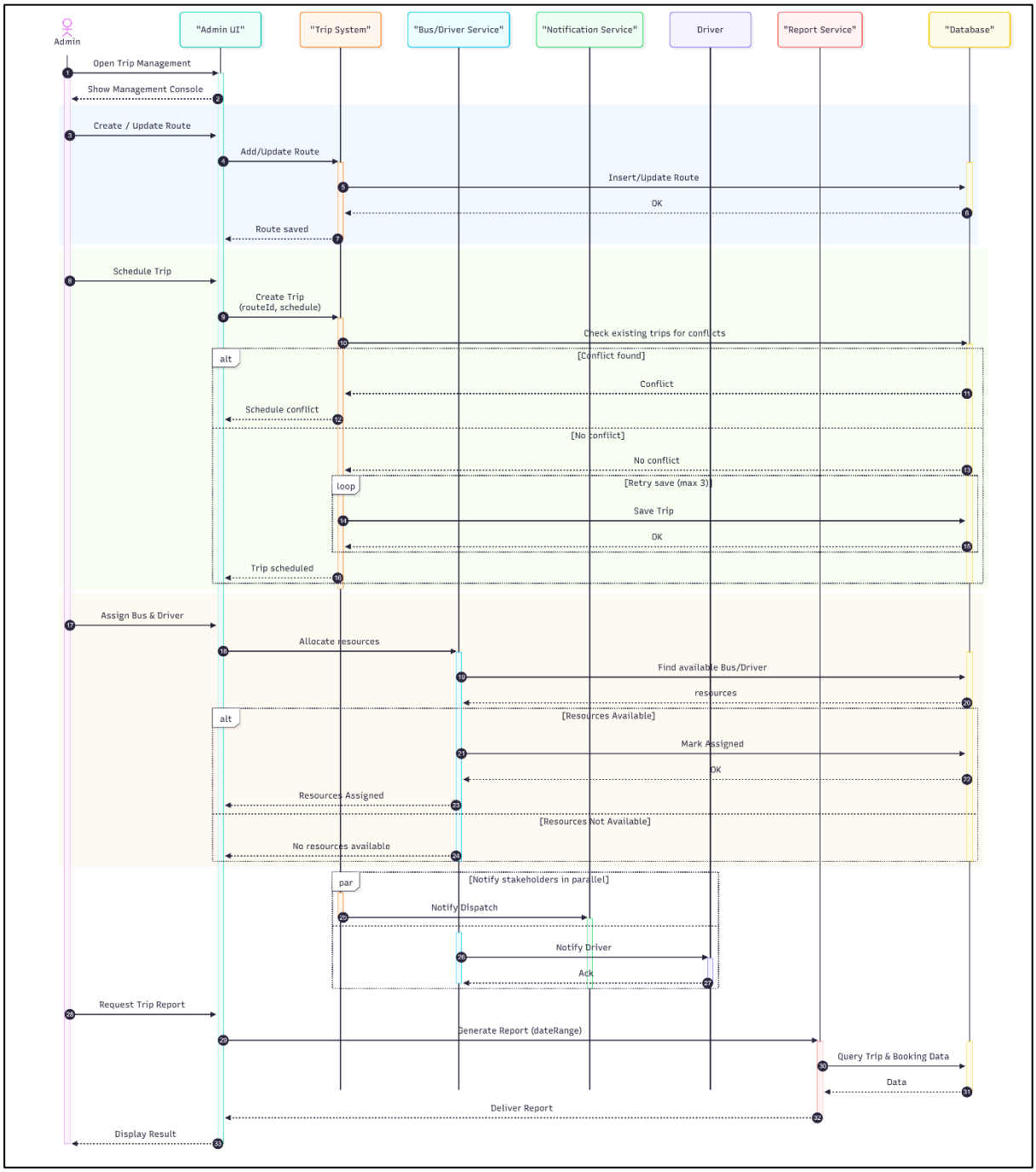


Sequence Diagrams

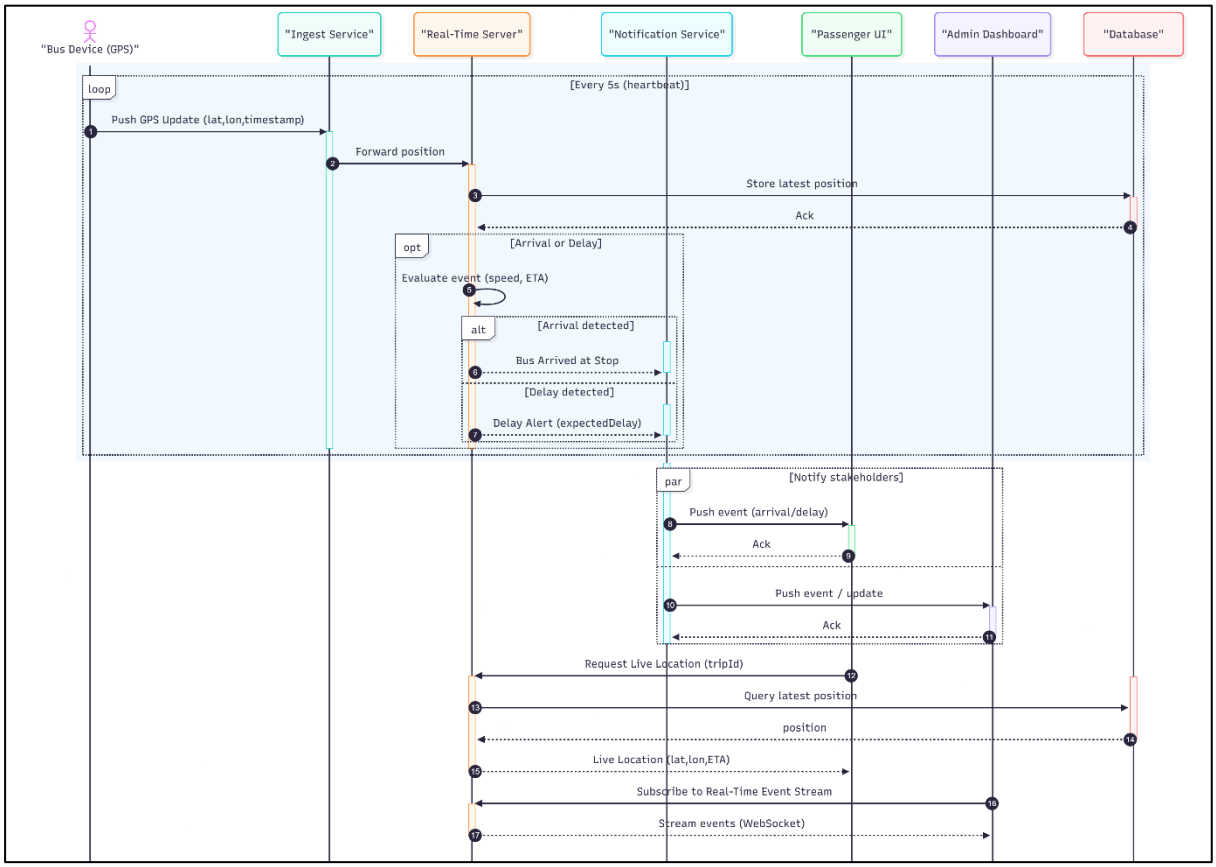
Passenger Trip Booking



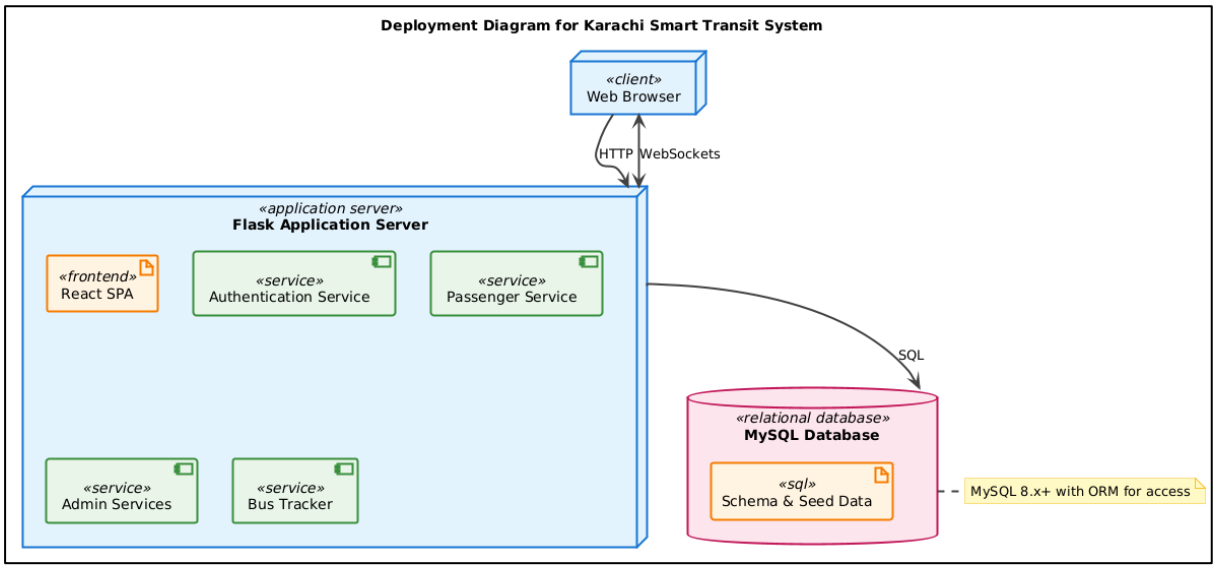
Admin Trip Management



Real-Time Bus Tracking - Notification Flow

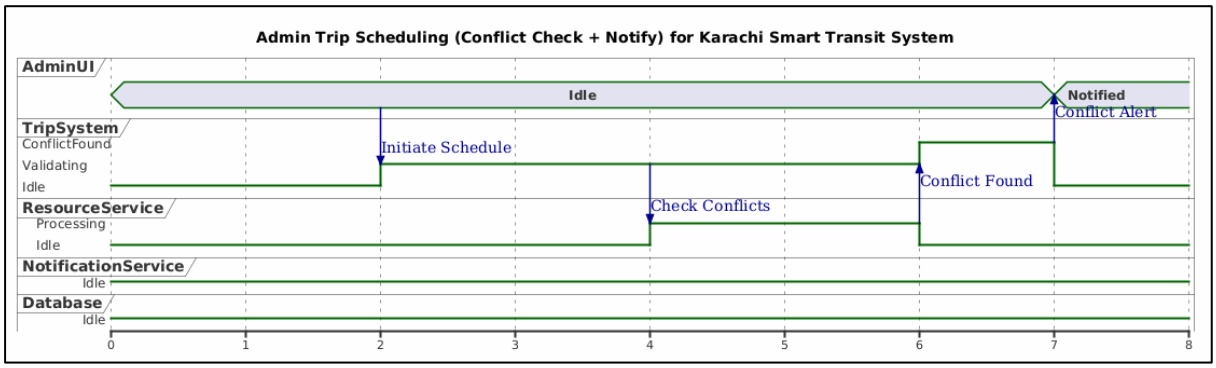


Deployment Diagram

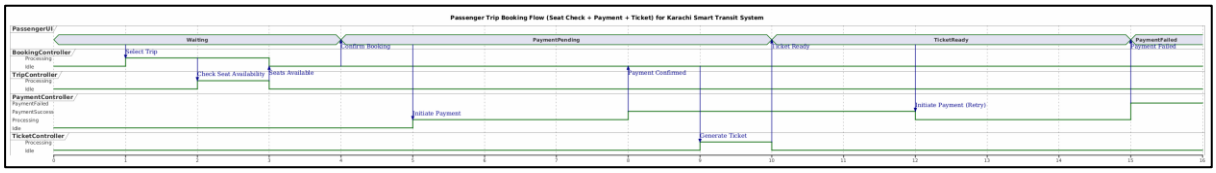


Timing Diagram

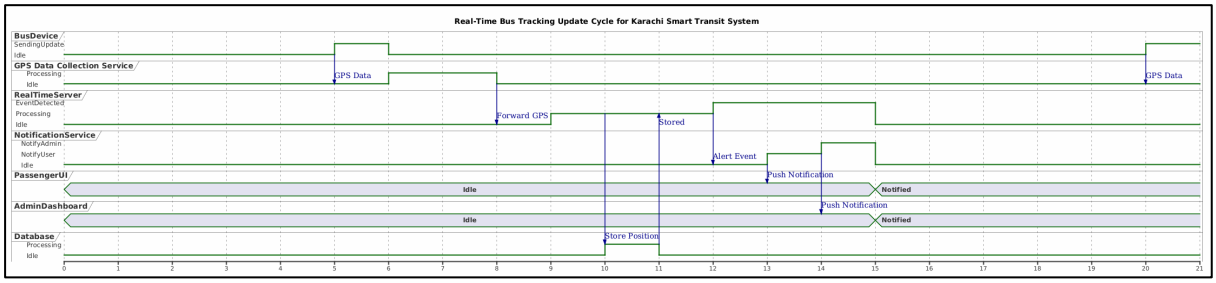
Admin Trip Scheduling



Passenger Trip Booking Flow



Real-Time Bus Tracking Update Cycle



Architectural View (4+1 Model)

Logical View

(Refer to the Class Diagram included above for structural representation.)

- **Core Classes and Responsibilities**:
  - **User (Abstract Base Class)**: Represents authenticated entities with attributes like userId, name, email, phone, and password. Provides methods for login() and logout(). Serves as the root for role-based subclasses.
  - **Passenger (Inherits from User)**: Handles passenger-specific actions, including viewRoutes(), bookTrip(), and viewTicket(). Manages bookings and ticket retrieval.
  - **Driver (Inherits from User)**: Tracks driver availability (availability: bool) and license (licenseNo). Includes methods like viewAssignedTrips() and updateTripStatus() for real-time updates.
  - **Admin (Inherits from User)**: Oversees system management with methods such as manageRoutes(), assignBus(), and generateReports(). Responsible for scheduling and resource allocation.
  - **Route**: Defines t[illegible] oint, endPoint, and methods like addStop() an[illegible] ltiple Stop instances.
  - **Stop**: Represents individual stops with stopId, stopName, and location.
  - **Bus**: Models vehicles with busId, plateNo, capacity, and status.
  - **Trip**: Encapsulates scheduled journeys with tripId, departureTime, arrivalTime, status, and updateStatus(). Associated with one Bus and one Driver.
  - **Booking**: Links passengers to trips, including bookingId, seatNo, status, and methods confirm() and cancel(). Generates Ticket and includes Payment.
  - **Ticket**: Digital tickets with ticketId, issueDate, and qrCode for verification.
  - **Payment**: Handles transactions with paymentId, amount, method, status, and processPayment(). Processed via PaymentProcessor (interface implemented by Bank).
  - **Report**: Generated by admins with reportId, type, and generate() for analytics.

- **Relationships**:
  - **Inheritance**: Passenger, Driver, and Admin inherit from User.
  - **Associations**:
    - Route includes multiple Stop (composition)
    - Route has multiple Trip (aggregation)
    - Trip uses one Bus and is assigned one Driver
    - Passenger makes multiple Booking
    - Booking is for one Trip, generates multiple Ticket, and includes one Payment
    - Payment is processed by PaymentProcessor
    - Admin manages Route, Driver, Bus, and Trip, and generates Report.
  - **Exceptions**: SeatUnavailableException and PaymentFailedException handle booking and payment failures.

Process View

(Refer to the Sequence Diagrams above for runtime behavior of booking, trip management, and real-time tracking.)

Flask-based multi-request architecture ensures each HTTP request is processed independently.

- **Concurrency and Critical Sections**:
  - **Booking Process**: Involves critical sections for seat availability checks and payment processing. When a passenger books a trip, the system queries the database for available seats (critical read operation). Payment retries (up to 3 attempts) use loops with status checks to handle failures, ensuring atomicity via database transactions. If seats are unavailable, the process terminates without locking resources.
  - **Trip Management**: The admin's scheduling operation checks for conflicts (e.g., overlapping trips) within a critical section and performs retries when committing updates to ensure consistency. Resource assignment (bus/driver) involves availability queries and updates, serialized to prevent double-booking.
  - **Real-Time Tracking**: GPS updates from bus devices occur every 5 seconds in a background loop, processed concurrently without blocking user interactions. Event detection (e.g., arrival/delay) triggers notifications, with parallel pushes to passengers and admins via WebSockets.

- **Background Tasks and Synchronization**:
  - Notifications run in parallel (e.g., notifying drivers and dispatching alerts during trip assignment). Real-time server evaluates events asynchronously, storing positions and firing alerts without halting the main flow.
  - **Synchronization:** Database operations use locks for writes (e.g., updating trip status or payment logs). WebSocket streams for admin dashboards and passenger UIs ensure real-time updates without polling, reducing load.

Development View

- **Technology Stack**:
  - **Backend**: Flask (Python web framework) for RESTful APIs and WebSocket handling.
  - **Frontend**: React (with HTML/JS) for single-page application (SPA) interfaces, enabling dynamic UIs for passengers and admins.
  - **Database**: MySQL with ORM (e.g., SQLAlchemy) for object-relational mapping, ensuring type safety and query efficiency.
  - **Additional Tools**: WebSockets for real-time features; payment integration via external processors.

- **Project Structure**:
  - **app.py:** Main Flask application entry point, configuring routes, middleware, and server startup.
  - **models/:** Database models (e.g., User, Trip, Booking) using ORM classes, mirroring the class diagram.
  - **controllers/:** Route handlers (e.g., BookingController, TripController) for HTTP/WebSocket endpoints, processing requests and responses.
  - **services/:** Business logic layers (e.g., PaymentService, NotificationService) for operations like payment processing and event detection.
  - **templates/:** HTML templates for server-side rendering if needed, though primarily React-driven.
  - **static/:** Static assets (CSS, JS, images) served by Flask.
  - **database/:** Migration scripts, seed data, and connection configurations for MySQL.

- **Layering**:
  - **Presentation Layer**: React components for UIs (e.g., booking forms, admin dashboards).
  - **Application Layer**: Flask controllers handling requests, integrating with services.
  - **Domain Layer**: Services encapsulating business rules (e.g., seat allocation, report generation).
  - **Data Layer**: ORM models and database interactions, with repositories for queries.

Physical View

(Refer to the Deployment Diagram provided above for server-tier and network topology.)

- **3-Tier Architecture**:
  - **Client Tier**: Web browsers running React SPAs, communicating via HTTP for requests and WebSockets for real-time updates (e.g., bus tracking notifications).
  - **Application Tier**: Flask application server hosting controllers, services, and ORM. Handles business logic, authentication, and integrations (e.g., payment processors).
  - **Data Tier**: MySQL database server storing schema and data, accessed via SQL queries through the ORM.

- **Deployment Specifics**:
  - **Communication:** HTTP for standard interactions (e.g., booking); WebSockets for bidirectional real-time streams (e.g., GPS updates to passengers/admins).
  - **Scalability:** Flask servers can be horizontally scaled with load balancers; MySQL supports clustering/replication for high availability. Bus devices (GPS) push data to the ingest service, which forwards to the real-time server.
  - **Security:** Authentication service verifies users; data encrypted in transit (HTTPS/WebSockets over TLS).

+1 (Use Case View)

(See the Use Case Diagram inserted earlier for actor–system interactions.)

- **Actors**:
  - **Passenger**: End-user interacting with the system for transit needs.
  - **Admin**: System administrator managing operations.
  - **Driver**: On-field personnel verifying tickets and receiving assignments.
  - **Payment Service**: External system handling transactions.
  - **Authentication Service**: Handles login/verification.

- **Key Use Cases**:
  - **Passenger**:
    - Register/Login
    - Search Trips
    - Book Trip
    - Cancel Booking
    - Make Payment
    - View Ticket
  - **Admin**:
    - Manage Routes/Buses/Drivers
    - Schedule Trips
    - Generate Reports
  - **Driver**:
    - Log In
    - Verify Tickets
    - View Assigned Trips

References

About the Unified Modeling Language Specification Version 2.5.1. (n.d.). https://www.omg.org/spec/UML

Contributors, M. O. J. T. a. B. (n.d.). *Bootstrap*. https://getbootstrap.com/

draw.io Documentation. (n.d.). https://www.drawio.com/doc/

Kruchten, P. (1995). Architecture Blueprints---The "4+1" view model of software architecture. *IEEE Software 12 (6)*, 540–555. https://doi.org/10.1145/216591.216611

Mermaid. (n.d.). https://mermaid.js.org/intro/syntax-reference.html

MySQL :: MySQL Documentation. (n.d.). https://dev.mysql.com/doc/

PlantUML Language Reference Guide. (n.d.). PlantUML.com. https://plantuml.com/guide

Quick Start – React. (n.d.). https://react.dev/learn

RFC 6455: The WebSocket Protocol. (n.d.). IETF Datatracker. https://datatracker.ietf.org/doc/html/rfc6455

SQLAlchemy Documentation — SQLAlchemy 2.0 Documentation. (n.d.). https://docs.sqlalchemy.org/en/20/

Welcome to Flask — Flask Documentation (3.1.x). (n.d.). https://flask.palletsprojects.com/en/stable/